

Name :- Mihir Mungara

Snowflake Assignment-11 : Create a Clone of an Existing Database Using Zero-Copy Cloning

1. Create a Clone of an Existing Database Using Zero-Copy Cloning

Objective

Create a clone of an existing database without physically copying the data and verify that changes in the clone do not affect the original database.

Step 1: Create Original Database and Table

```
CREATE OR REPLACE DATABASE SALES_DB;
```

```
USE DATABASE SALES_DB;
```

```
USE SCHEMA PUBLIC;
```

```
CREATE OR REPLACE TABLE PRODUCTS (  
    PRODUCT_ID INT,  
    PRODUCT_NAME STRING,  
    PRICE NUMBER(10,2)  
);
```

```
INSERT INTO PRODUCTS VALUES  
(1, 'Laptop', 50000),  
(2, 'Mobile', 25000),  
(3, 'Tablet', 15000);
```

Verify:

```
SELECT * FROM PRODUCTS;
```

	# PRODUCT_ID	△ PRODUCT_NAME	# PRICE
1	1	Laptop	50000.00
2	2	Mobile	25000.00
3	3	Tablet	15000.00

Step 2: Create Database Clone

```
CREATE OR REPLACE DATABASE SALES_DB_CLONE
CLONE SALES_DB;
```

Verify databases:

```
SHOW DATABASES;
```

Step 3: View Data in Clone

```
USE DATABASE SALES_DB_CLONE;

SELECT * FROM PUBLIC.PRODUCTS;
```

Output:

PRODUCT_ID	PRODUCT_NAME	PRICE
1	Laptop	50000
2	Mobile	25000
3	Tablet	15000

Step 4: Modify Data in Clone

```
UPDATE PUBLIC.PRODUCTS
SET PRODUCT_NAME = 'Gaming Laptop'
WHERE PRODUCT_ID = 1;
```

Verify:

```
SELECT * FROM PUBLIC.PRODUCTS;
```

Output:

	PRODUCT_ID	PRODUCT_NAME	PRICE
1	1	Gaming Laptop	50000.00
2	2	Mobile	25000.00
3	3	Tablet	15000.00

Step 5: Verify Original Database

```
USE DATABASE SALES_DB;  
  
SELECT * FROM PUBLIC.PRODUCTS;
```

Output:

	# PRODUCT_ID	A PRODUCT_NAME	# PRICE
1	1	Laptop	50000.00
2	2	Mobile	25000.00
3	3	Tablet	15000.00

Observation

Changes made in the cloned database do not affect the original database because Snowflake uses Zero-Copy Cloning.

2. Set Up a Stream and Task for Continuous Integration

Objective

Capture data changes using Streams and automatically process them using Tasks.

Step 1: Create Source Table

```
CREATE OR REPLACE TABLE EMPLOYEES (  
    EMP_ID INT,  
    EMP_NAME STRING,  
    SALARY NUMBER  
);
```

Insert data:

```
INSERT INTO EMPLOYEES VALUES  
(1, 'Alice', 50000),  
(2, 'Bob', 60000);
```

Step 2: Create Target Table

```
CREATE OR REPLACE TABLE EMPLOYEE_AUDIT (  
    EMP_ID INT,  
    EMP_NAME STRING,  
    SALARY NUMBER  
);
```

Step 3: Create Stream

```
CREATE OR REPLACE STREAM EMPLOYEE_STREAM  
ON TABLE EMPLOYEES;
```

Verify:

```
SHOW STREAMS;
```

Step 4: Generate Changes

```
INSERT INTO EMPLOYEES VALUES  
(3, 'Charlie', 70000);
```

```
INSERT INTO EMPLOYEES VALUES  
(4, 'David', 65000);
```

View Stream Data:

```
SELECT * FROM EMPLOYEE_STREAM;
```

output :-

⌂	# EMP_ID	A EMP_NAME	# SALARY	A METADATA\$ACTION	0 1 METADATA\$ISUPDATE	A METADATA\$ROW_ID
1	3	Charlie	70000	INSERT	FALSE	4e0980cb8acdd89f795e4813e44a7968f0b0cbd1
2	4	David	65000	INSERT	FALSE	9affa265bbdc1f5b1161186a17c80038b0782e93

Step 5: Create Task

```
CREATE OR REPLACE TASK EMPLOYEE_TASK
WAREHOUSE = COMPUTE_WH
SCHEDULE = '5 MINUTE'
AS
INSERT INTO EMPLOYEE_AUDIT
SELECT EMP_ID,
       EMP_NAME,
       SALARY
FROM EMPLOYEE_STREAM
WHERE METADATA$ACTION = 'INSERT';
```

Step 6: Enable Task

```
ALTER TASK EMPLOYEE_TASK RESUME;
```

Verify:

```
SHOW TASKS;
```

Step 7: Monitor Task Execution

Task status:

```
SHOW TASKS;
```

Task history:

```
SELECT *
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY())
ORDER BY SCHEDULED_TIME DESC;
```

Step 8: Verify Target Table

After task execution:

```
SELECT * FROM EMPLOYEE_AUDIT;
```

Output:

EMP_ID	EMP_NAME	SALARY
4	David	65000
3	Charlie	70000

Observation

The Stream captured changes and the Task automatically loaded them into the target table.

3. Implement a Simple UDF in SQL

Objective

Create a User Defined Function (UDF) to calculate product price including tax.

Step 1: Create Products Table

```
CREATE OR REPLACE TABLE PRODUCTS (  
    PRODUCT_ID INT,  
    PRODUCT_NAME STRING,  
    PRICE NUMBER(10,2)  
);
```

Insert data:

```
INSERT INTO PRODUCTS VALUES  
(1, 'Laptop', 50000),  
(2, 'Mobile', 25000),  
(3, 'Tablet', 15000);
```

Step 2: Create SQL UDF

```
CREATE OR REPLACE FUNCTION ADD_TAX(PRICE NUMBER(10,2))  
RETURNS NUMBER(10,2)  
AS  
$$  
PRICE * 1.10  
$$;
```

Verify:

```
SHOW USER FUNCTIONS;
```

Step 3: Use UDF in Query

```
SELECT  
    PRODUCT_NAME,  
    PRICE,  
    ADD_TAX(PRICE) AS PRICE_WITH_TAX  
FROM PRODUCTS;
```

Output:

ID	PRODUCT_NAME	PRICE	PRICE_WITH_TAX
1	Laptop	50000.00	55000.000
2	Mobile	25000.00	27500.000
3	Tablet	15000.00	16500.000

Advantages of UDF

1. Reusable business logic.
2. Simplifies complex calculations.
3. Improves query readability.
4. Centralized maintenance of calculation logic.

Conclusion

Successfully implemented:

1. Zero-Copy Cloning to create a database clone without duplicating storage.
2. Streams and Tasks for continuous data processing and automation.
3. SQL User Defined Functions (UDFs) to encapsulate reusable business logic.